

LAB-01: Packet Capture Fundamentals

Table of Contents

1. Objective	4
2. Lab Environment	4
3. Concepts Primer.....	4
What a NIC does with frames not addressed to it:.....	4
What libpcap/npcap is vs what Wireshark is:	4
Capture filter vs display filter:	5
4. Setup	5
Task 1: Baseline capture	6
What was done:.....	6
What is in the screenshot:	6
Why is both directions displayed:.....	6
Why this matters:	6
Task 2 Promiscuous mode test.....	8
What was done:.....	8
Result with promiscuous mode OFF:	8
What "Network unreachable" on Metasploitable2 means:	8
Important limitation	8
Result with promiscuous mode ON:.....	9
Task 3: Capture filter vs display filter.....	12
Part A — Capture filter icmp:	12
Part B — No capture filter, display filter icmp:.....	12
The key distinction:.....	12
When to use which in practice:	12
Task 4: tcpdump vs Wireshark	15
Command used:	15
Breaking down the command:	15
What happened:	15
What the screenshot shows:	16
Why this matters:	16
Where .pcap file is saved:	16
Task 5 : Layer-by-layer inspection.....	18

What was done:	18
Frame layer (Layer 1/2 boundary):	18
Ethernet II layer (Layer 2 — Data Link):	19
Internet Protocol v4 layer (Layer 3 — Network):	19
ICMP layer (Layer 4 — in practice Layer 3.5, since ICMP is technically IP-adjacent):	19
The encapsulation concept this demonstrates	19

1. Objective

This lab investigates how packet capture actually works at the interface level — what Wireshark is doing when it "listens", why promiscuous mode matters, what the difference is between a capture filter and a display filter, and how Wireshark and tcpdump relate to each other. The goal is to build a solid working mental model of what a packet capture is actually capturing before using it as the primary analysis tool in every subsequent lab.

2. Lab Environment

- Both VMs on Internal Network (intnet) :chosen because it produces clean, isolated traffic with zero background noise from the host's real network. Every packet visible in Wireshark is one you generated.
- Kali Linux: attacker/analysis machine runs Wireshark and tcpdump
- Metasploitable2: target/traffic generator enerates pings and SSH attempts toward Kali

3. Concepts Primer

What a NIC does with frames not addressed to it:

Every Ethernet frame carries a destination MAC address. Normally a NIC operates in non-promiscuous mode: the hardware itself checks each incoming frame's destination MAC, and if it doesn't match the NIC's own MAC (and isn't a broadcast), the NIC drops the frame before the operating system ever sees it. This is hardware-level filtering the CPU and OS are never involved, which is why it's efficient.

In promiscuous mode the NIC is instructed to pass ALL frames up to the OS regardless of destination MAC including unicast traffic addressed to other devices. This is how a packet capture tool can see traffic that isn't addressed to the machine running it.

What libpcap/npcap is vs what Wireshark is:

These are two separate things that most people mentally merge:

- libpcap (Linux) / npcap (Windows) is the low-level capture library. It puts the NIC into promiscuous mode, receives raw frames from the kernel, and writes them to a buffer or file. It does the actual capturing.

- Wireshark is a GUI application that reads what libpcap/npcap collects and renders it in a human-readable, color-coded, filterable interface. Wireshark is not capturing anything itself — it's a display layer on top of libpcap.
- tcpdump is also just a command-line interface to the same libpcap library. This is why Wireshark can open a .pcap file saved by tcpdump they both wrote data using the same underlying format from the same library.

Capture filter vs display filter:

This is one of the most practically important distinctions in Wireshark:

- A **capture filter** is applied at the libpcap level, before packets are written to memory or disk. Packets that don't match the filter are discarded immediately and never stored. Syntax uses BPF (Berkeley Packet Filter) syntax e.g. icmp, host 192.168.56.10, port 80. These are efficient (less memory/disk use) but cannot be changed while a capture is running, and you cannot go back and see traffic you filtered out.
- A **display filter** is applied after packets are already captured and stored in memory. It hides non-matching packets from view but does not delete them. You can add, change, or remove display filters at any time without affecting the underlying capture. Syntax is Wireshark's own protocol-aware language e.g. icmp, tcp.port == 22, arp. Far more powerful and flexible than capture filters, but all traffic still hits memory first.

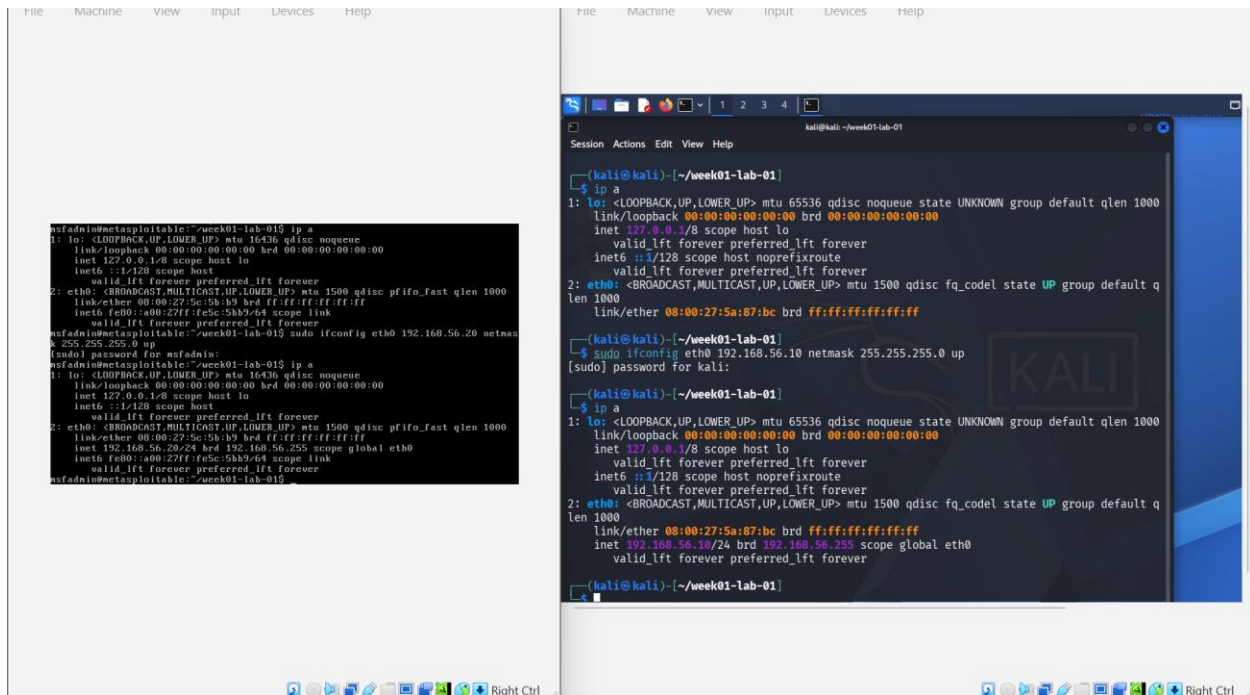
The practical rule: use capture filters when you know exactly what you're looking for and you're on a noisy network. Use display filters during analysis when you want to slice and dice a capture you've already taken.

4. Setup

Both VMs were placed on Internal Network (intnet). Static IPs were verified:

- Kali: 192.168.56.10
- Metasploitable2: 192.168.56.20

Wireshark was opened on Kali and eth0 was selected as the capture interface.



Task 1: Baseline capture

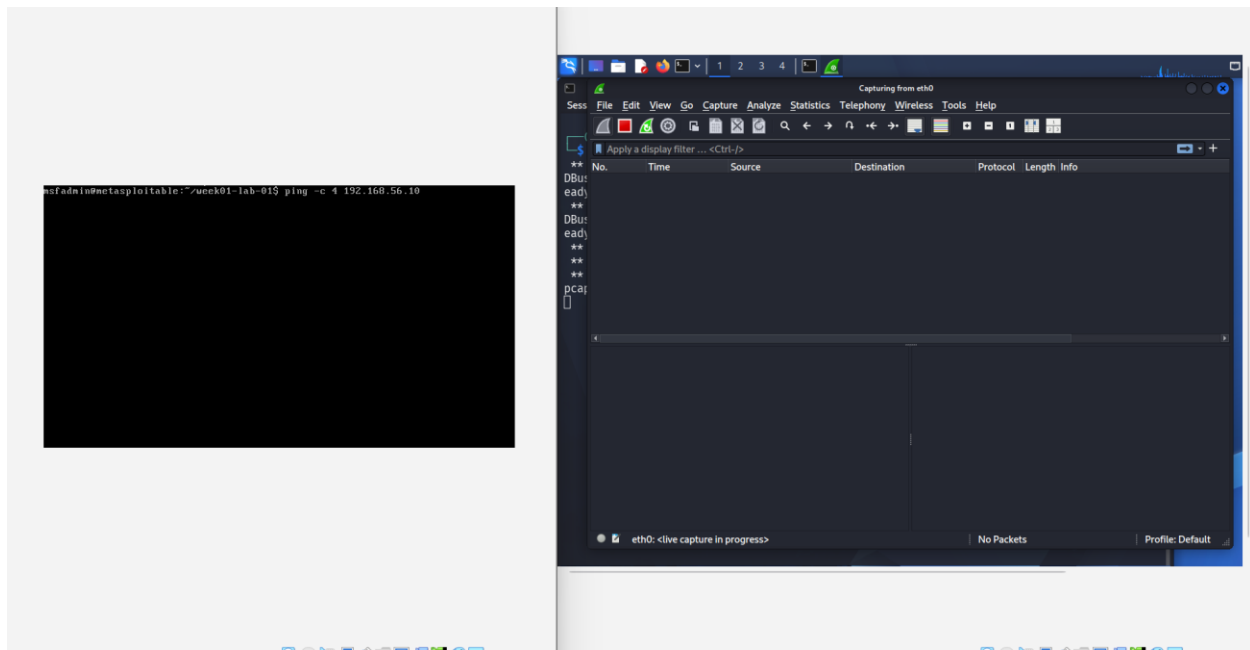
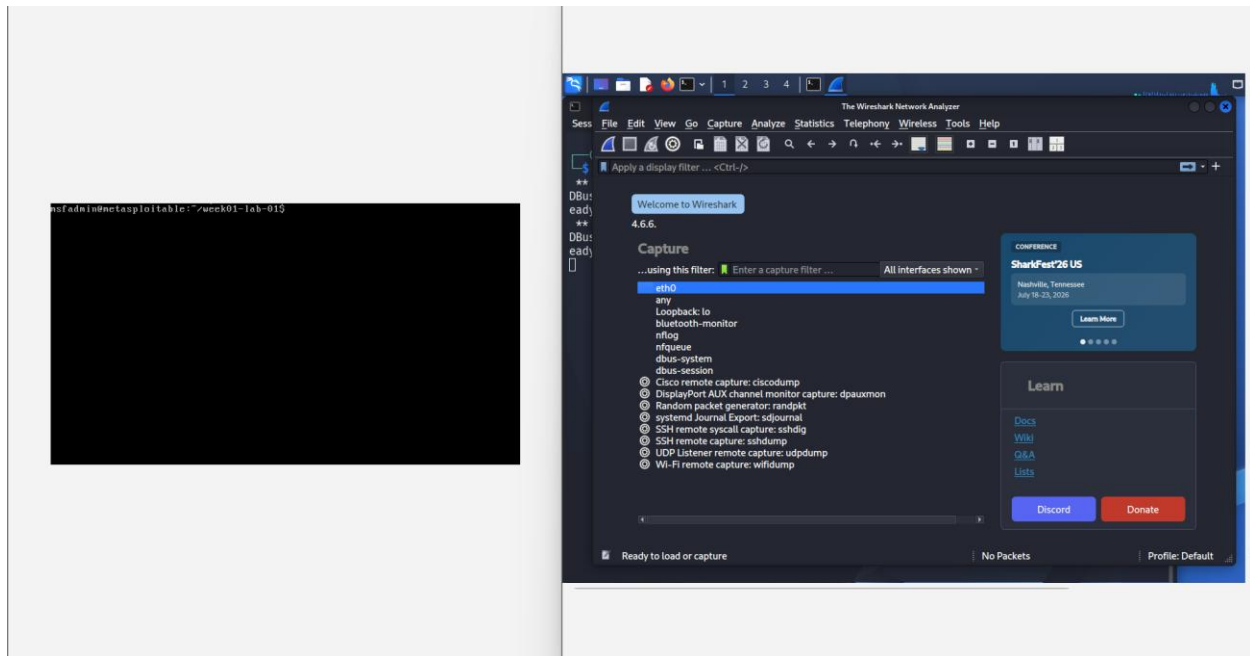
What was done: A Wireshark capture was started on Kali's eth0 with no filter applied. From Metasploitable2, ping -c 4 192.168.56.10 was run targeting Kali. The capture was stopped and inspected.

What is in the screenshot: Both ICMP Echo Request (Type 8) packets AND ICMP Echo Reply (Type 0) packets are visible in Wireshark. The requests appear as packets sourced from Metasploitable2 (192.168.56.20) destined for Kali (192.168.56.10). The replies are the reverse.

Why is both directions displayed: Wireshark captures all traffic passing through eth0 both inbound frames (arriving at Kali) and outbound frames (leaving Kali). When Metasploitable2 sends an ICMP Request to Kali, Kali receives it (inbound captured). Kali then generates an ICMP Reply and sends it back (outbound also captured). Eth0 is the single point through which all of this traffic flows, so Wireshark sees all of it.

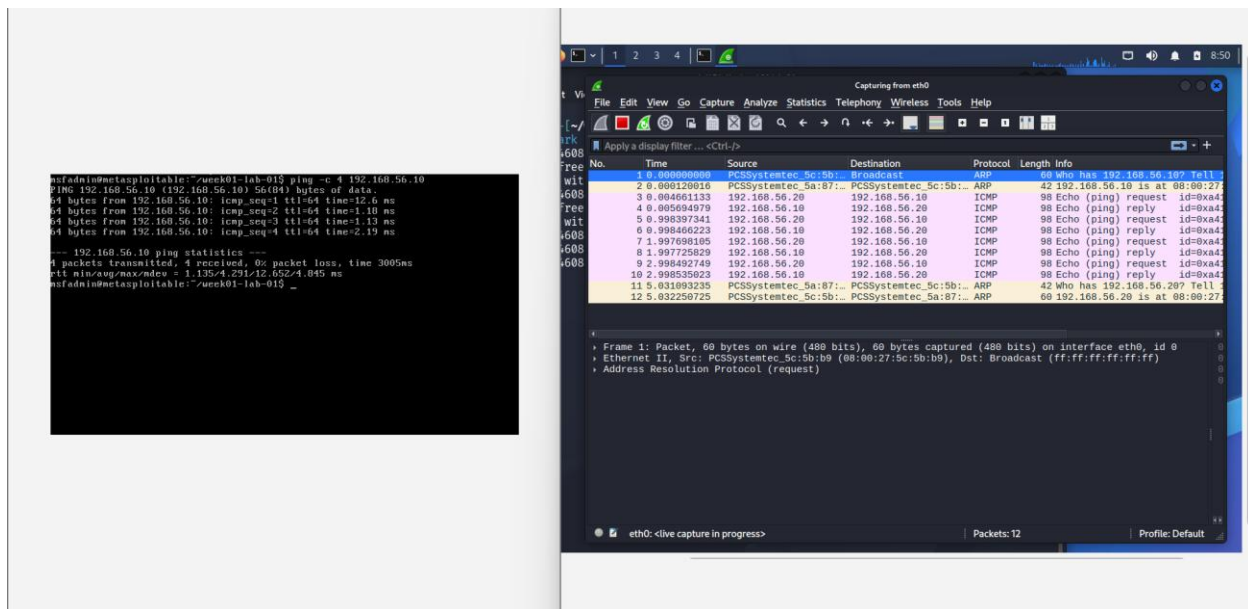
Why this matters: This tells you that packet capture on an interface is bidirectional by default — you are capturing everything flowing through that interface in both directions, not just traffic addressed to you.

Start a Wireshark capture on Kali with no filter.



Stop capture. do you see both the ICMP request AND reply?

Yesssss! They are visible as attached in the screenshot as well



Task 2 Promiscuous mode test

What was done: Promiscuous mode was disabled in Wireshark's capture options. A capture was started on Kali. From Metasploitable2, ping -c 4 192.168.56.45 was run (an unused IP within the same subnet — not Kali itself). The capture was inspected.

Result with promiscuous mode OFF:

In a 2-VM Internal Network setup, you will still see the ARP broadcast traffic — because broadcast frames (ff:ff:ff:ff:ff:ff destination MAC) are always received by every NIC regardless of promiscuous mode setting. ARP is a broadcast protocol. So even without promiscuous mode, Kali's NIC accepts ARP request frames because they're addressed to everyone. What you will NOT see is any unicast traffic between two other devices that isn't addressed to Kali — but since there are only 2 VMs, there is no "other" traffic to miss.

What "Network unreachable" on Metasploitable2 means:

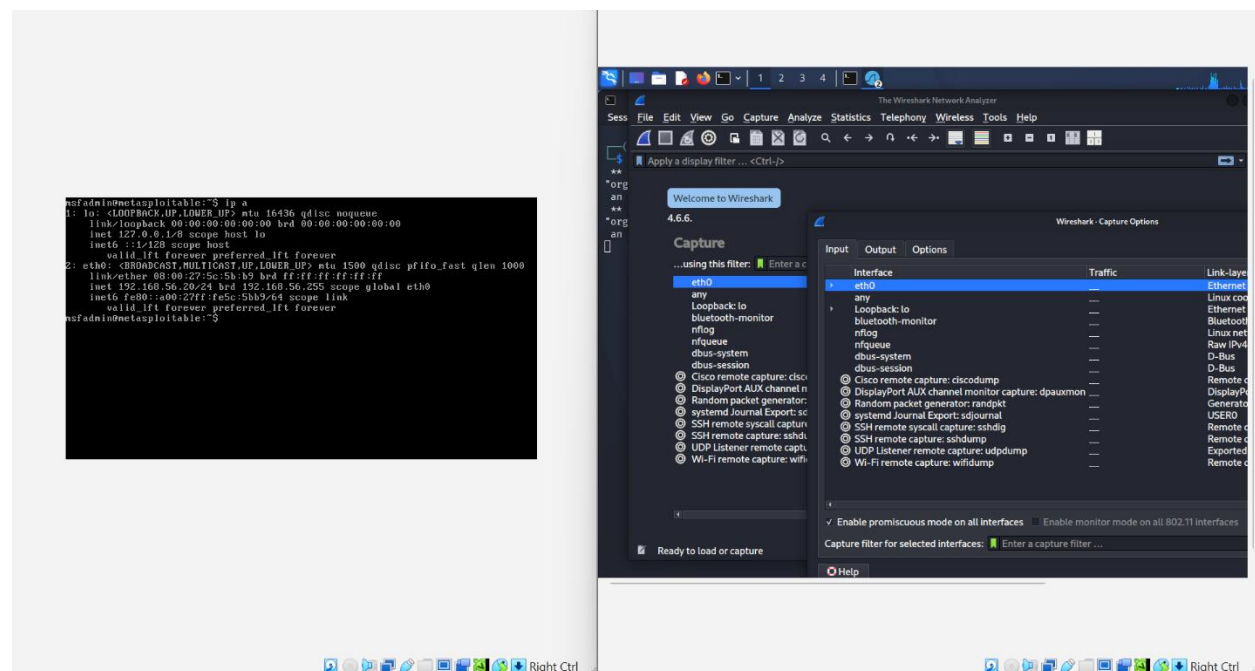
When Metasploitable2 tried to ping 192.168.56.45, its kernel checked the routing table. The 192.168.56.0/24 network is local (same subnet), so the kernel attempted to ARP-resolve 192.168.56.45 before sending the ICMP packet. ARP broadcasts went out, nobody replied (no such host exists), and after ARP timeout, the ping reported "Destination Host Unreachable." Nothing was "Network unreachable" here — you actually saw ARP activity on Wireshark, which is exactly the point.

Important limitation : True promiscuous-mode-only behavior requires a third host generating unicast traffic to something other than Kali, so Kali could see it with promiscuous ON

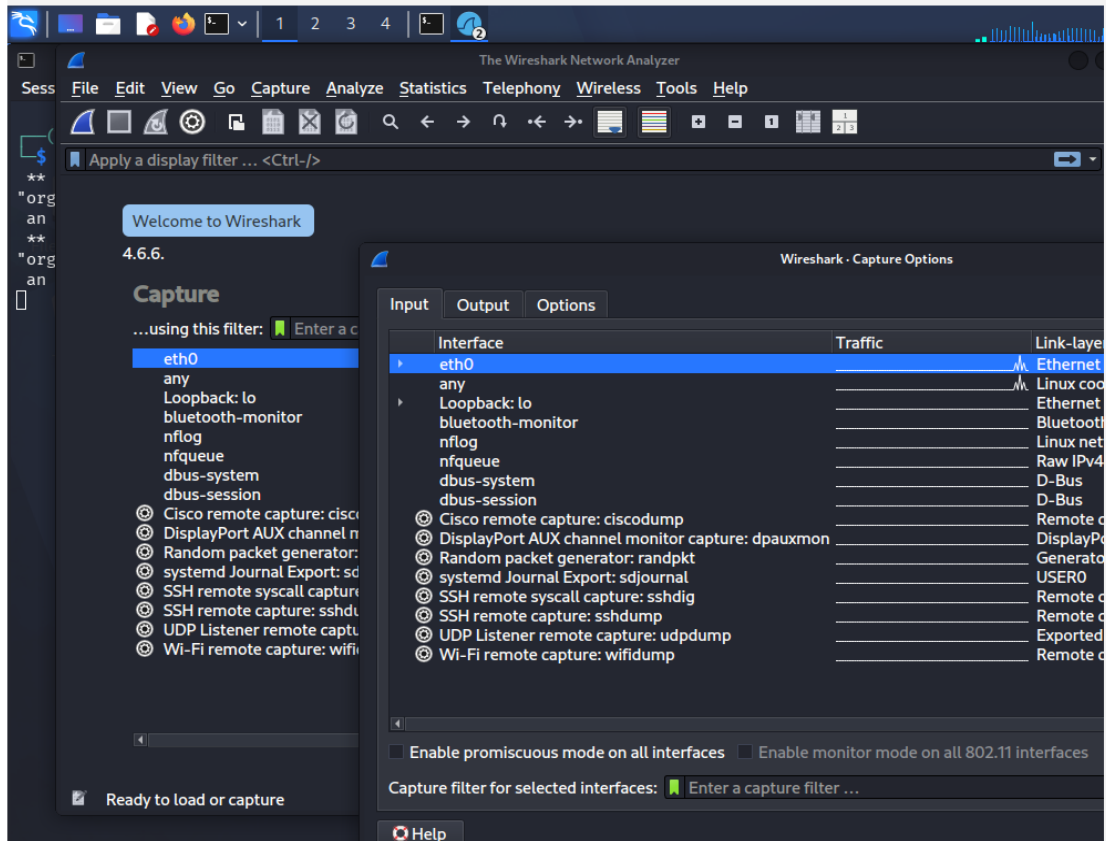
but not with it OFF. With only 2 VMs, ARP broadcasts serve as a proxy demonstration of the concept. This is an honest finding worth documenting in your report.

Result with promiscuous mode ON:

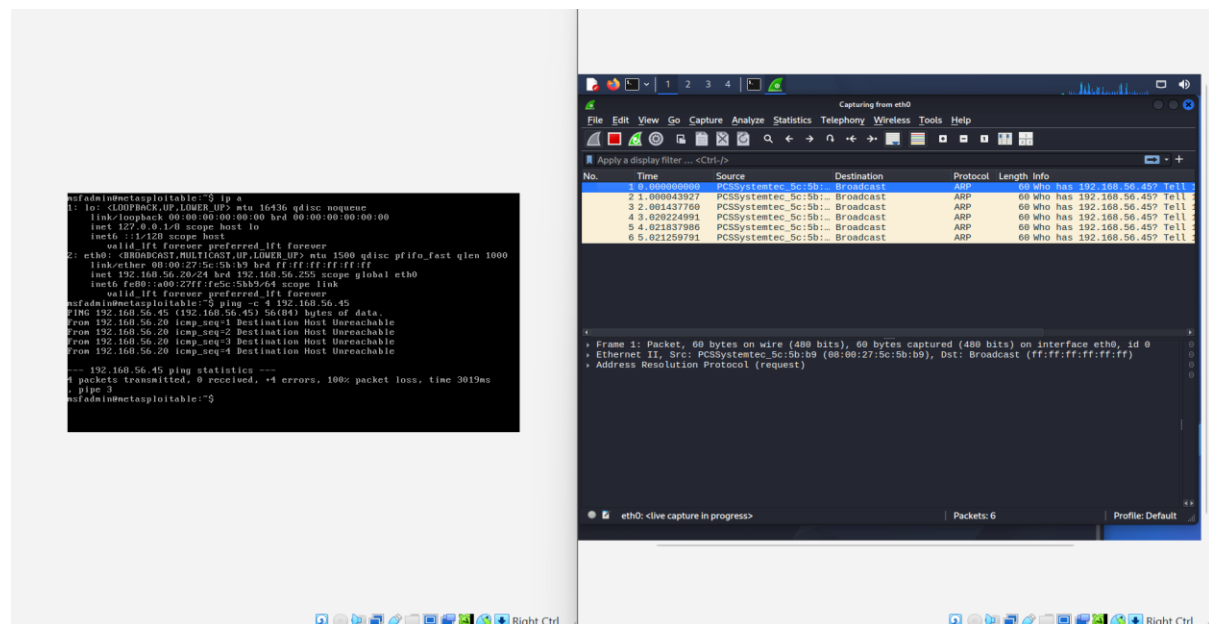
Same ARP traffic visible same result for a 2-VM lab. The conceptual difference is that with promiscuous ON, Kali's NIC would accept unicast frames addressed to other MAC addresses too. You cannot demonstrate this without a third VM.

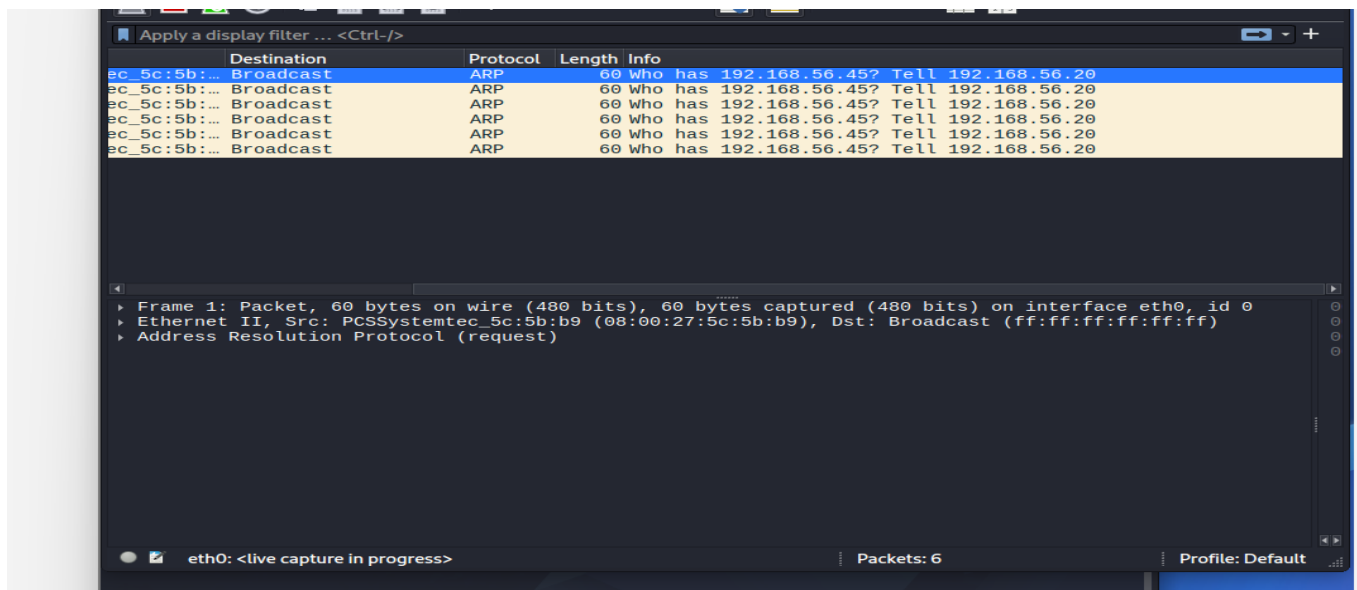


In Wireshark capture options, disable promiscuous mode. Start capture.

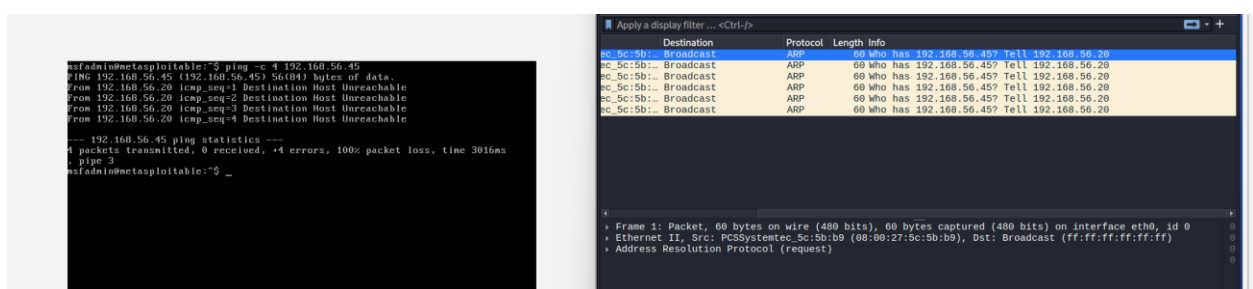
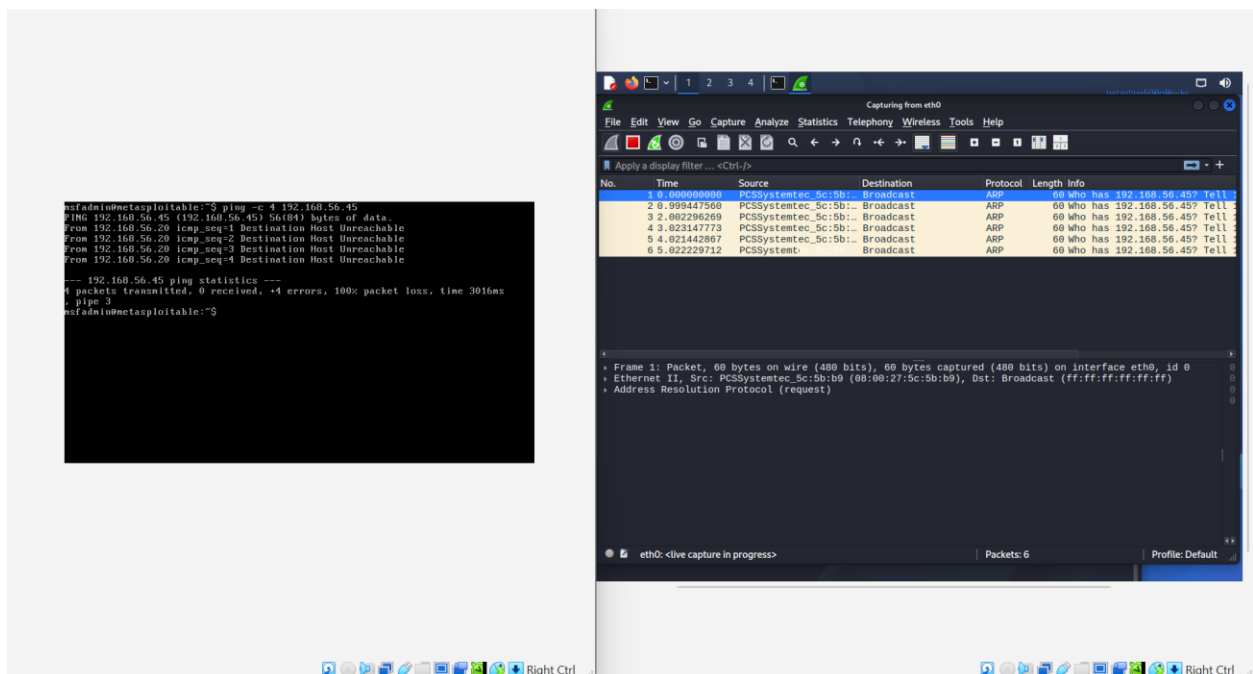


From Metasploitable2, ping some other address (doesn't need to succeed ping -c 4 192.168.56.45, some unused IP) — NOT Kali itself.





Enable promiscuous mode.



Task 3: Capture filter vs display filter

Part A — Capture filter icmp:

A capture filter of icmp was set before starting the capture. Mixed traffic was generated: a ping from Metasploitable2 AND an SSH connection attempt (ssh 192.168.56.10) AND ARP was allowed to happen naturally as part of those connections.

When the capture was stopped and the capture filter was cleared, only ICMP packets existed in the capture. No ARP, no TCP/SSH. This is because the capture filter instructed libpcap to discard any non-ICMP frame the moment it was received they were never written to memory. They are gone permanently from this capture. Clearing the display filter in Wireshark changes nothing because there is no hidden data everything that doesn't match icmp was thrown away at capture time.

Part B — No capture filter, display filter icmp:

A fresh capture was started with no capture filter. The same mixed traffic was generated. Then display filter icmp was applied in Wireshark.

Only ICMP packets are visible same visual result as Part A. But when the display filter is cleared, all the ARP and TCP/SSH packets reappear. They were captured and stored the whole time the display filter was only hiding them from view, not deleting them.

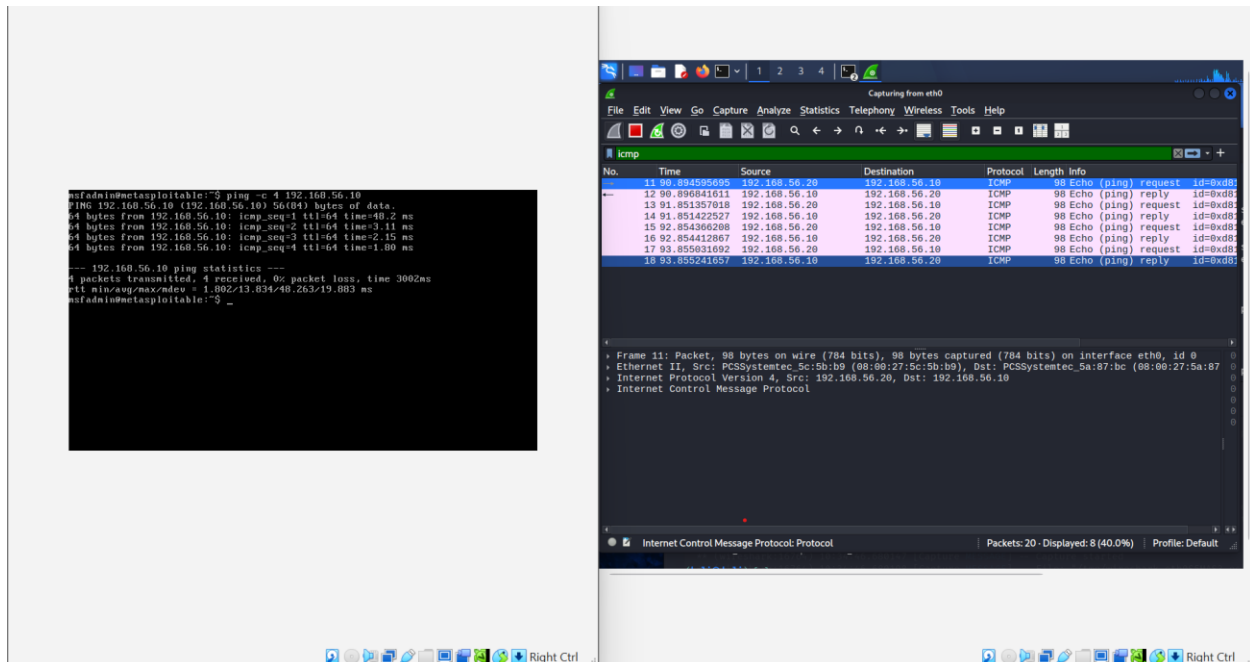
The key distinction: Capture filter = data is gone permanently. Display filter = data is hidden temporarily. Both can show you only ICMP, but only one of them preserves the rest.

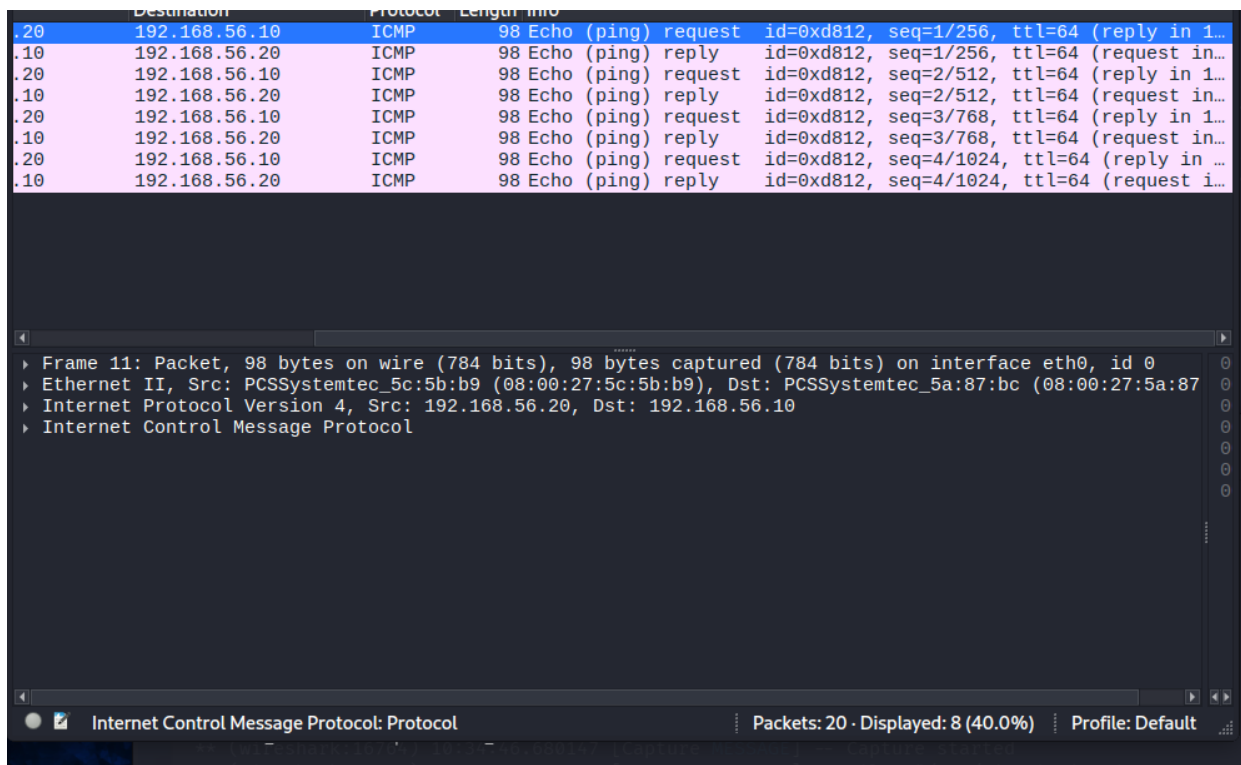
When to use which in practice:

- Running Wireshark on a busy production-like network where you only care about DNS traffic → capture filter port 53 — avoids filling memory with thousands of irrelevant packets per second.
- Doing post-capture analysis where you want to switch between viewing ARP, then ICMP, then TCP → display filters — switch freely, data is always there.

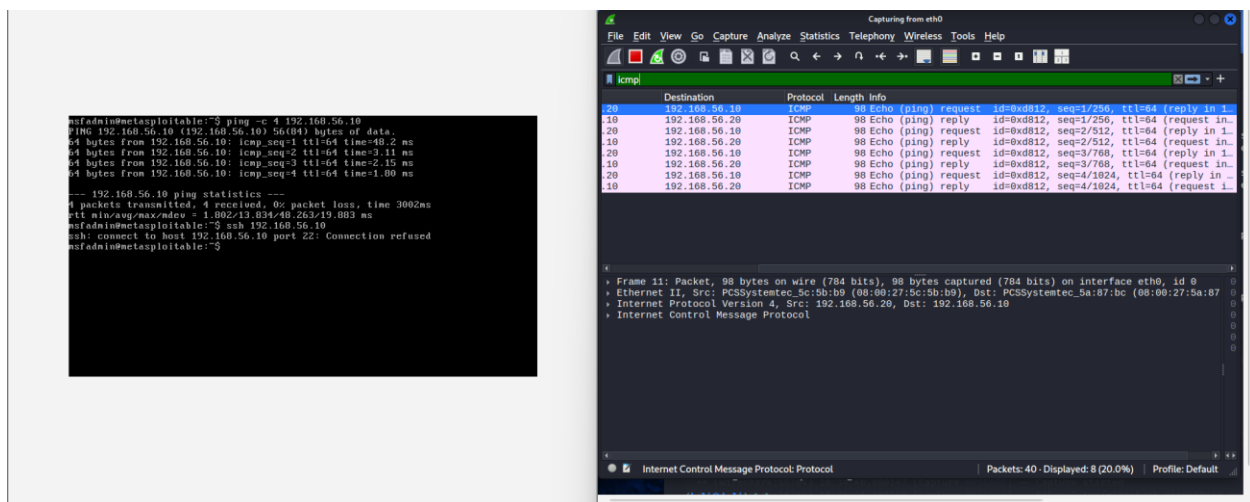
Set a **capture filter**: icmp. Start capture. From Metasploitable2, generate mixed traffic a ping AND an ssh connection attempt (ssh <kali-ip>, doesn't need to succeed) AND let ARP happen naturally.

Stop capture. Screenshot confirm ONLY icmp packets exist (try clearing any filter nothing else should appear, because capture filter discards at capture time).



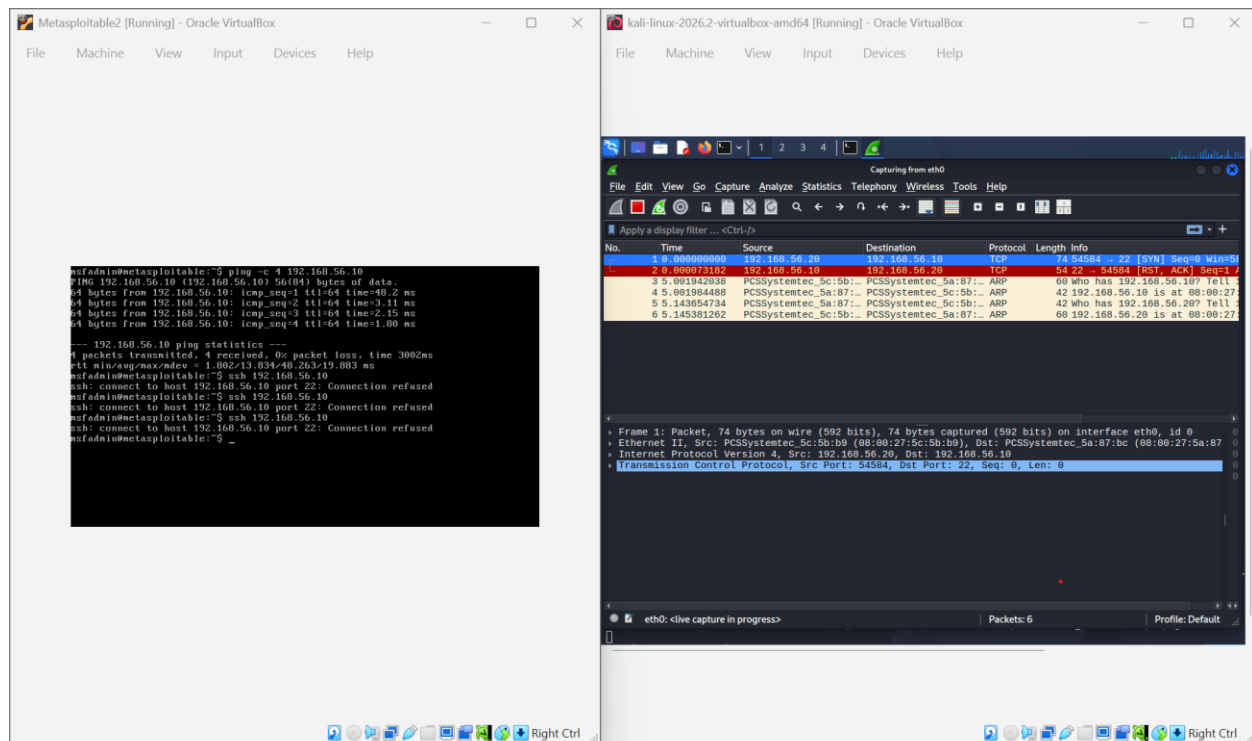


12. New capture, NO capture filter this time. Generate the same mixed traffic.



. Apply a **display filter**: icmp. Screenshot.

Clear the display filter. Screenshot confirm the ssh/arp packets are still there, just were hidden.



Task 4: tcpdump vs Wireshark

Command used:

bash

sudo tcpdump -i eth0 -w capture.pcap

Breaking down the command:

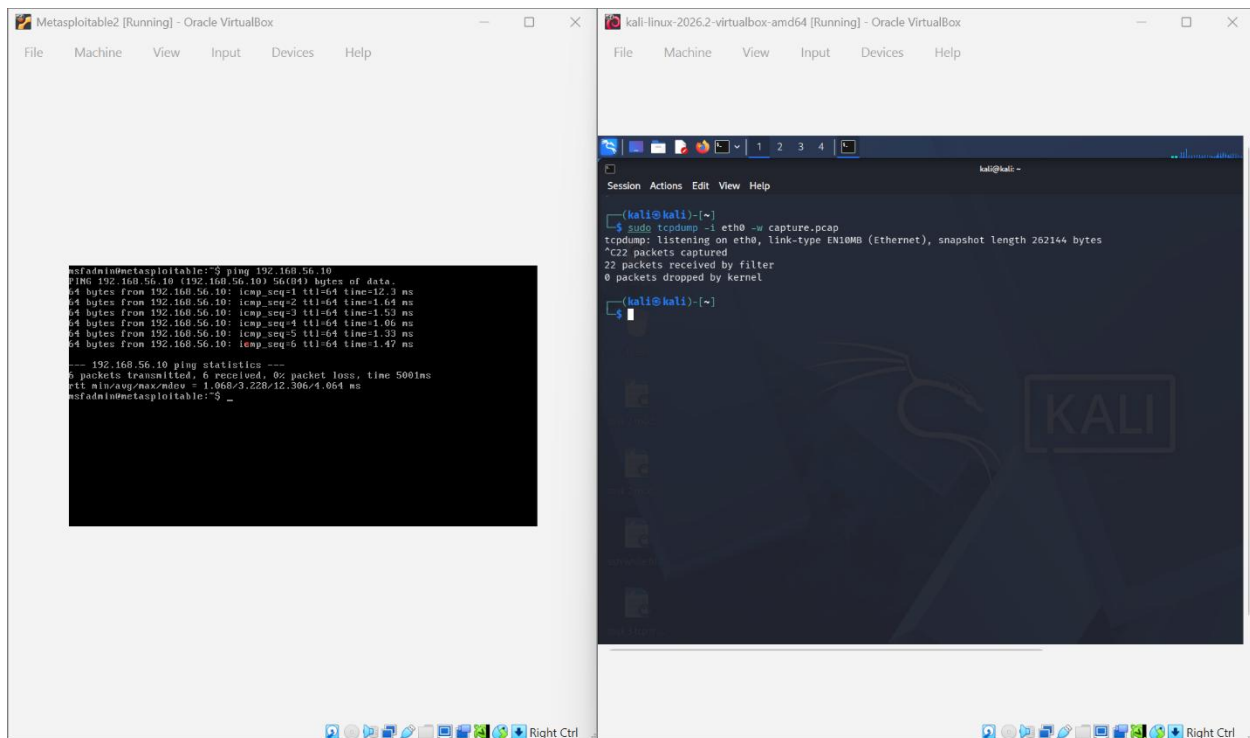
- sudo : tcpdump needs root privileges to put the NIC into capture mode and read raw frames from the kernel.
- tcpdump : the command-line packet capture tool. Sits on top of libpcap.
- -i eth0 : specifies which interface to capture on. eth0 is Kali's primary network interface connected to the Internal Network.
- -w capture.pcap : write captured packets to a file instead of printing them to the terminal. .pcap is the standard packet capture file format (also called pcap or pcapng for newer versions).

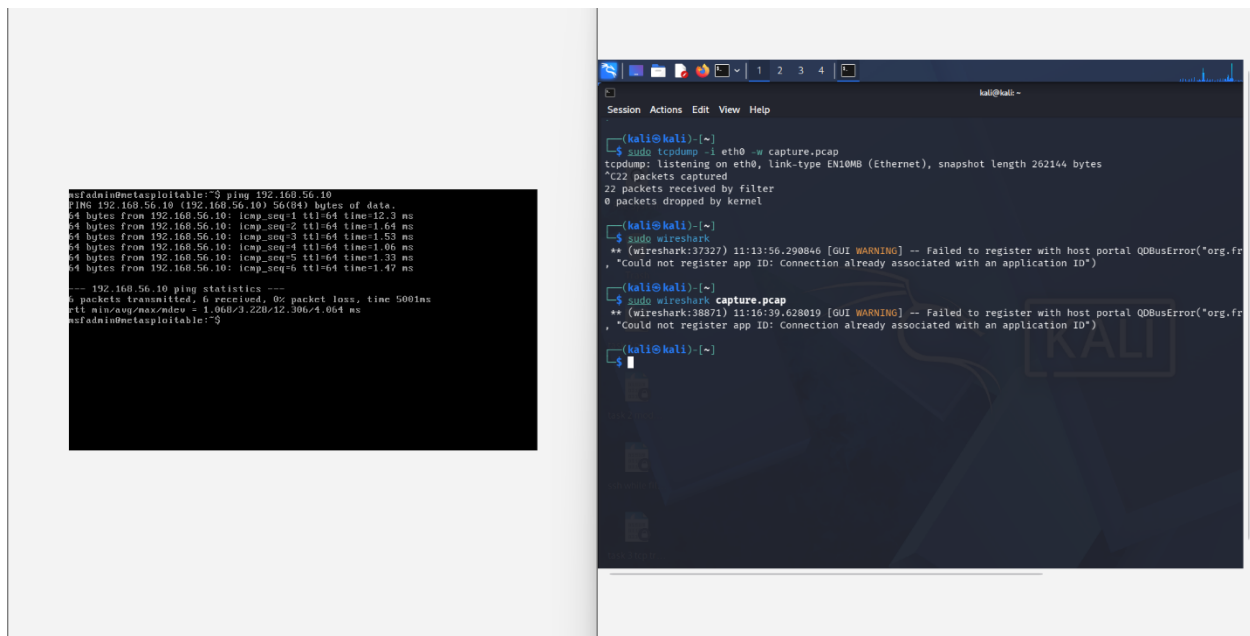
What happened: While tcpdump was running, a ping was generated from Metasploitable2. Ctrl+C stopped the capture. The file capture.pcap was then opened in Wireshark.

What the screenshot shows: The same ICMP packets visible in previous Wireshark captures are now visible when opening the tcpdump-saved file. The data is identical because both tcpdump and Wireshark use libpcap underneath — tcpdump saved raw frames in the standard pcap format, and Wireshark reads that same format perfectly.

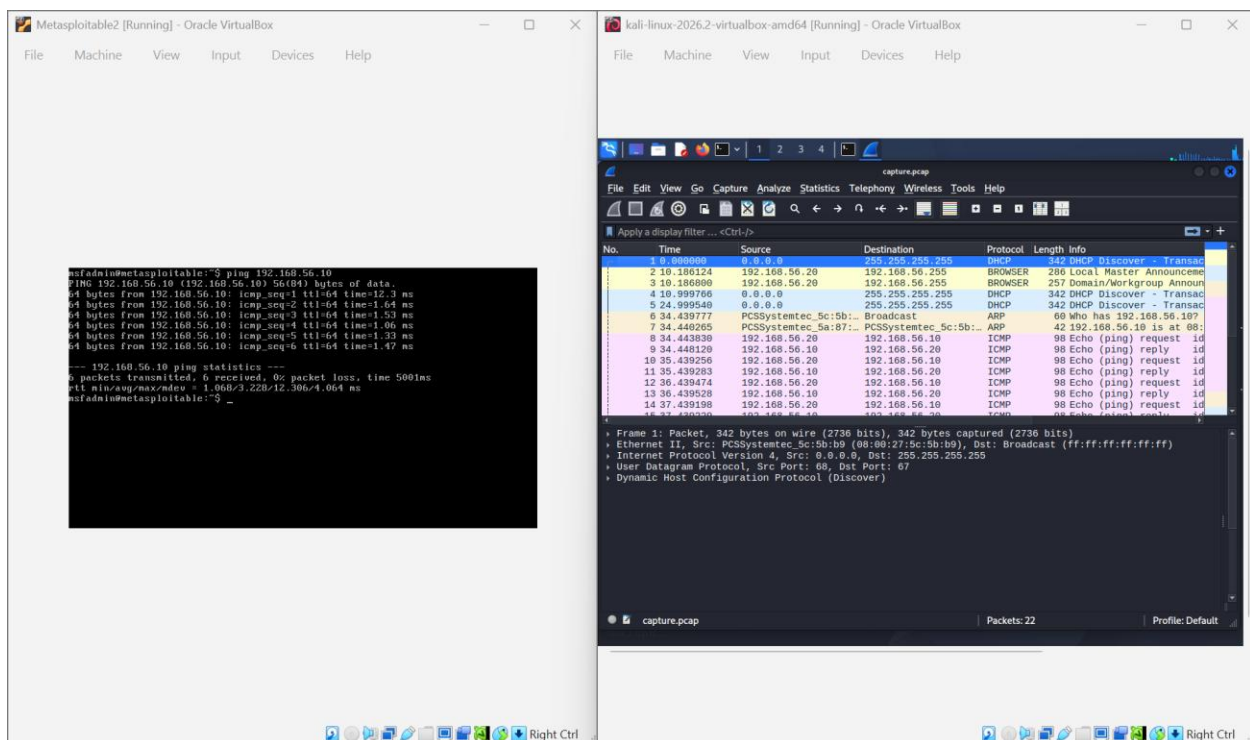
Why this matters: This proves Wireshark is not magic. It is a display tool. Any capture taken by tcpdump on a remote server, on a router, on any Linux/Unix system can be transferred to your analysis machine and opened in Wireshark for full visual analysis. In real incident response and network forensics, this workflow (capture with tcpdump remotely, analyze with Wireshark locally) is standard practice.

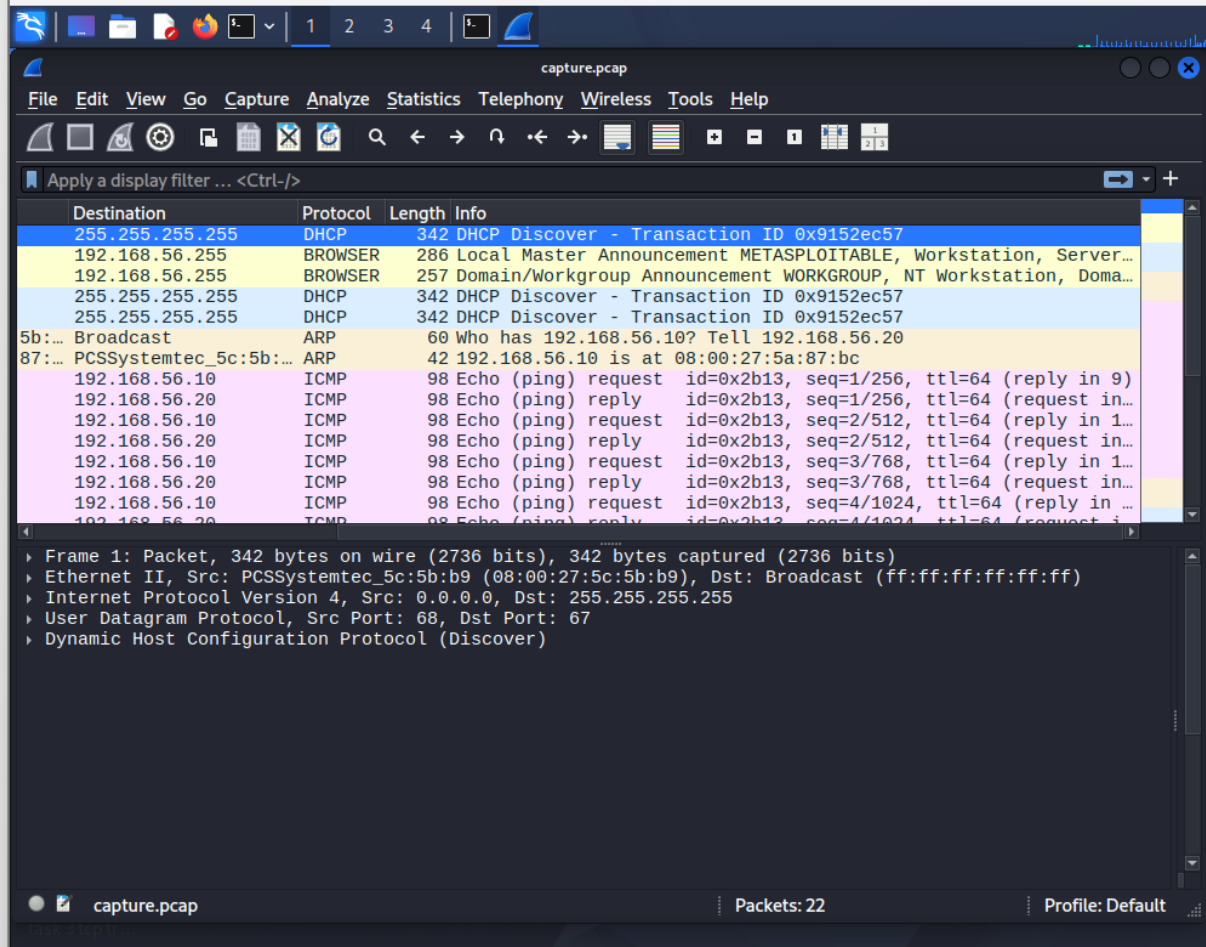
Where .pcap file is saved: By default, capture.pcap saves to the current working directory when you ran the command most likely /home/kali/ or /root/. Find it with `find / -name "*.pcap" 2>/dev/null`.





Open capture.pcap in Wireshark (wireshark capture.pcap or File → Open). Screenshot both the terminal capture and the opened file.





Task 5 : Layer-by-layer inspection

What was done: A single ICMP Echo Request packet from the baseline capture was selected and every layer was expanded in Wireshark's packet details pane.

Frame layer (Layer 1/2 boundary):

This is Wireshark's own metadata about the capture itself not data from the network. It records: which interface captured the packet, the timestamp (when it arrived), the total frame length in bytes, and what protocols were detected inside. This layer does not exist on the wire — Wireshark adds it when it writes the capture.

Ethernet II layer (Layer 2 — Data Link):

This is the actual frame header as it existed on the wire. It contains:

- Destination MAC address — the MAC of Kali's eth0 (since this packet was addressed to Kali)
- Source MAC address — the MAC of Metasploitable2's eth0
- EtherType 0x0800 — tells the receiving NIC that the payload inside this frame is an IPv4 packet

This is the layer that ARP operates at. ARP's entire job is to figure out what MAC address to put in this Destination field for a given IP address.

Internet Protocol v4 layer (Layer 3 — Network):

This is the IP header. It contains:

- Source IP: 192.168.56.20 (Metasploitable2)
- Destination IP: 192.168.56.10 (Kali)
- TTL (Time To Live): a counter that decrements by 1 at each router hop. Prevents packets from circulating forever. When TTL reaches 0, the router drops the packet and sends back an ICMP "Time Exceeded" message. The starting TTL value differs by OS — Linux defaults to 64, Windows to 128.
- Protocol field: 0x01 = ICMP, telling the receiving OS which Layer 4 protocol is inside

ICMP layer (Layer 4 — in practice Layer 3.5, since ICMP is technically IP-adjacent):

- Type 8 = Echo Request (ping sent). Type 0 = Echo Reply (ping answered).
- Code 0 : sub-classification within that type. For echo request/reply, code is always 0.
- Checksum : a value the sender computes over the ICMP header and data. The receiver recomputes it and compares if they don't match, the packet was corrupted in transit and is discarded.
- Identifier : a number assigned by the sender to match request/reply pairs, since multiple ping processes might be running simultaneously.
- Sequence number : increments by 1 with each ping. This is how ping counts "icmp_seq=1, 2, 3, 4" in its output and detects out-of-order or lost packets.

The encapsulation concept this demonstrates: Each layer wraps the one above it. The ICMP message is the payload inside the IP packet. The IP packet is the payload inside the Ethernet frame. On the wire, only the Ethernet frame exists as a complete unit. At each hop, routers strip the

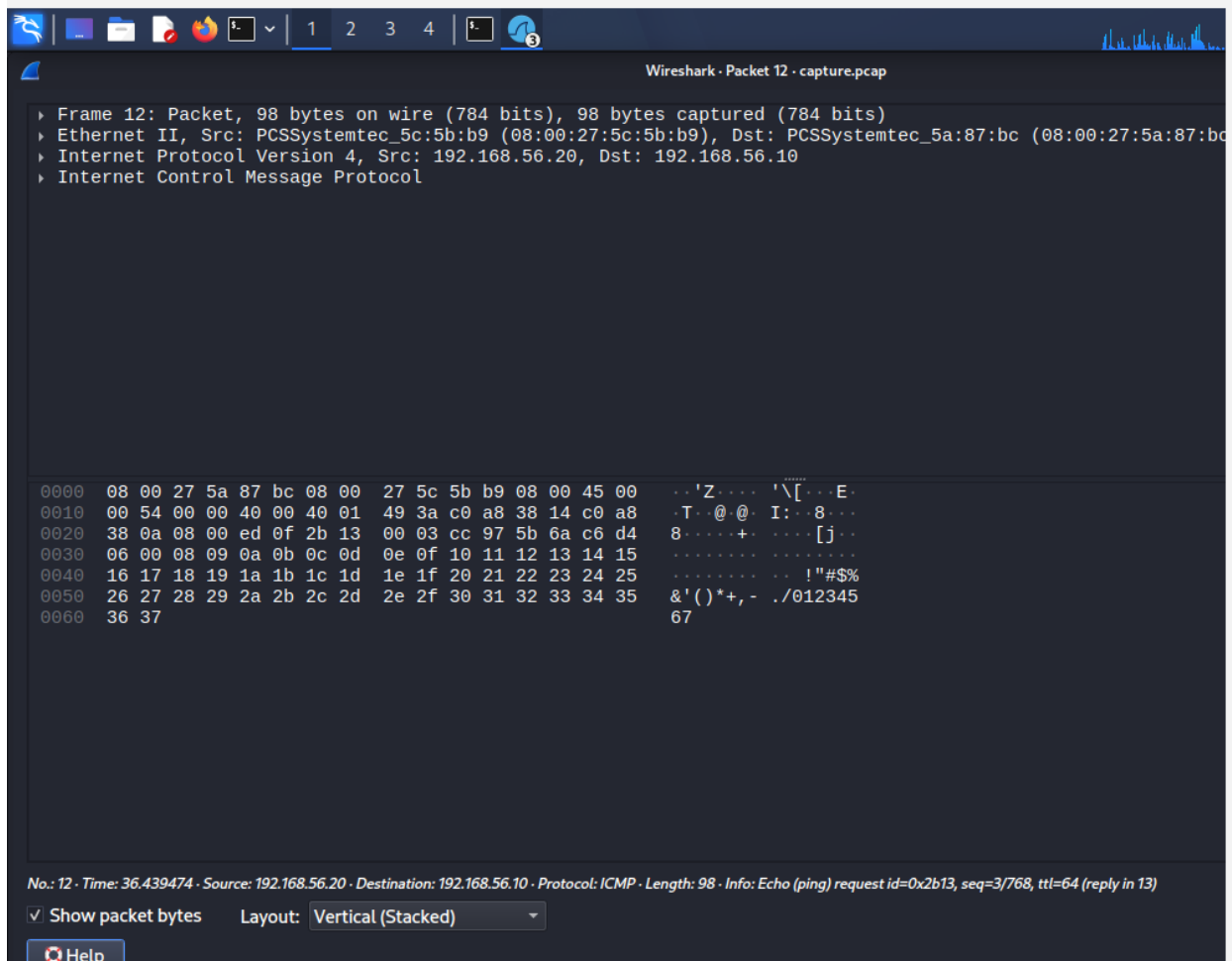
Ethernet header, read the IP header, make a routing decision, add a new Ethernet header for the next hop, and send it on. The IP and ICMP layers remain untouched across the entire path.

Task details:

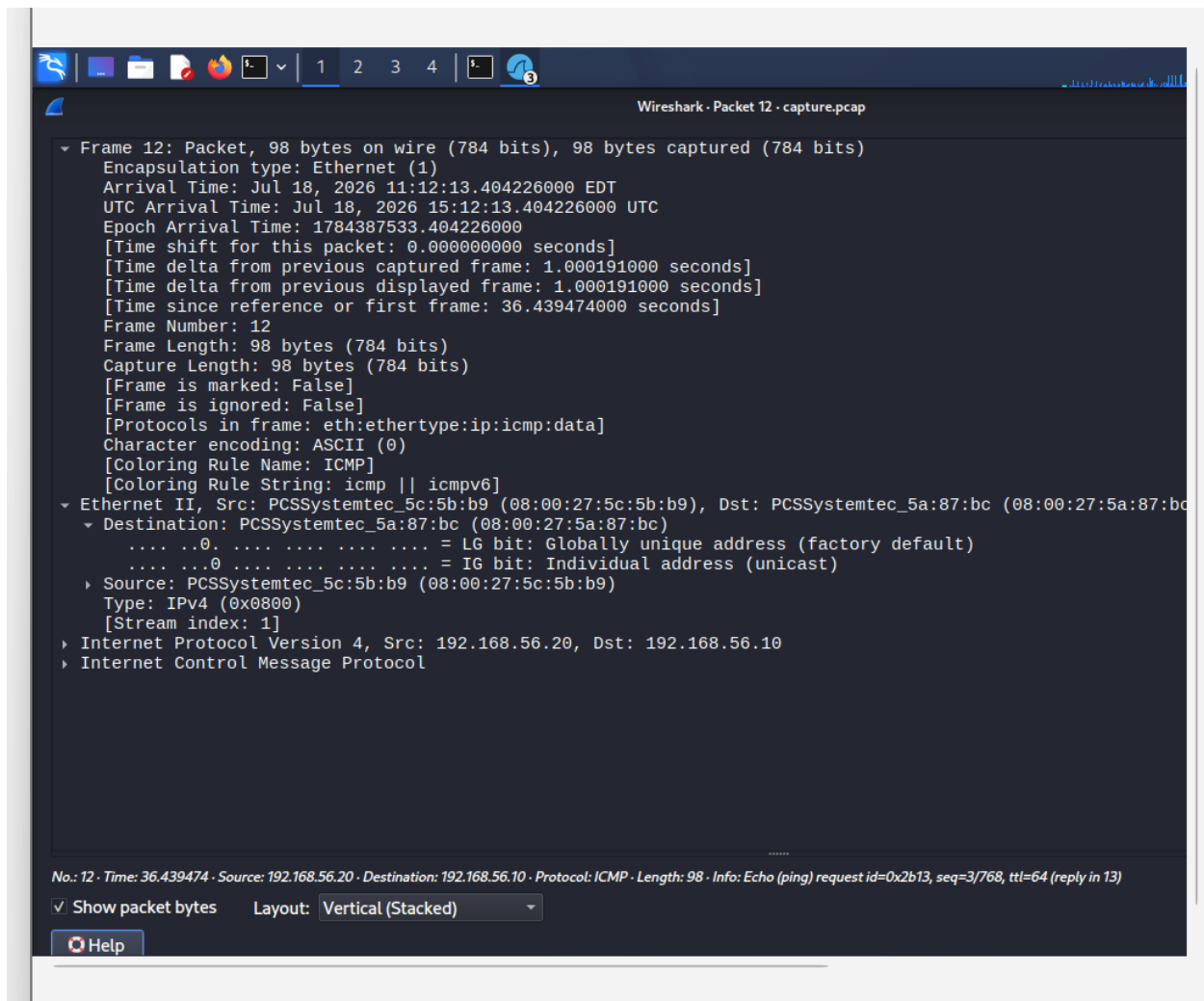
Pick any one ICMP packet from any capture above.

In the packet details pane, expand every layer: Frame → Ethernet II → Internet Protocol → ICMP.

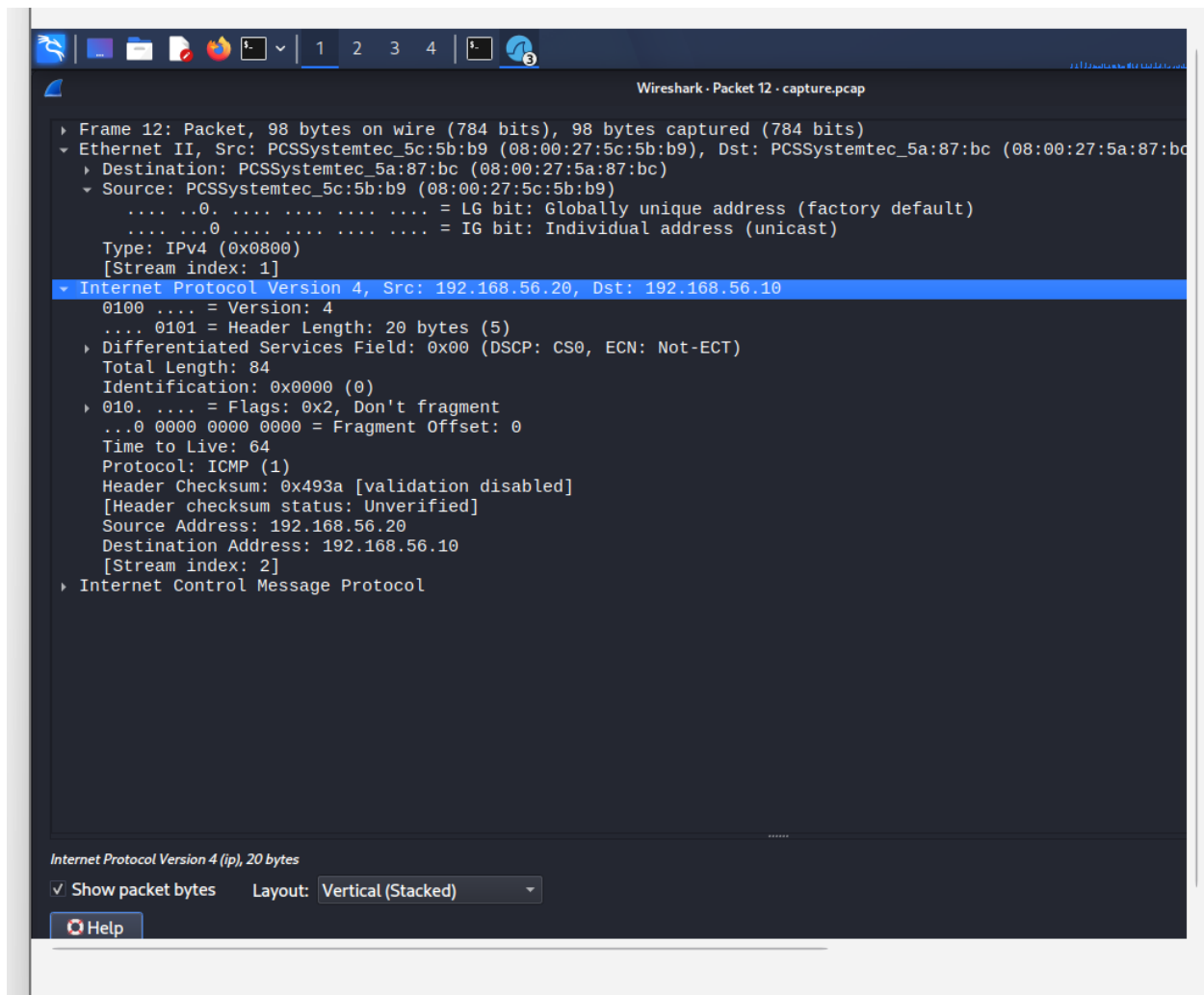
Screenshot fully expanded.



Frame detail:



IPV4 details



ICMP detail

▼ Internet Control Message Protocol

Type: Echo (ping) request (8)

Code: 0

Checksum: 0xed0f [correct]

[Checksum Status: Good]

Identifier (BE): 11027 (0x2b13)

Identifier (LE): 4907 (0x132b)

Sequence Number (BE): 3 (0x0003)

Sequence Number (LE): 768 (0x0300)

[Response frame: 13]

▶ ICMP Data: cc975b6ac6d4060008090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9fa0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9baabbcbdbddbe0e1e2e3e4e5e6e7e8e9eaebecedeeef

Internet Control Message Protocol (icmp), 64 bytes

✓ Show packet bytes

Layout: Vertical (Stacked)

Help